Async Credit Processing Service — Design

Architecture

The system has four runtime pieces:

1. **API (FastAPI)** — accepts credit requests, validates money and `user_id`, persists a pending credit, publishes work, returns immediately.
2. **Broker (RabbitMQ)** — durable handoff between API and workers. RabbitMQ provides durable task delivery; Celery uses late acknowledgments, prefetch 1, and explicit worker-loss rejection so interrupted work can be redelivered. Delivery is at least once; MongoDB transactions and idempotent claims protect against duplicate execution.
3. **Worker (Celery)** — applies the credit exactly once against MongoDB.
4. **MongoDB (single-node replica set)** — source of truth for credit lifecycle and user balances. A replica set is required so multi-document transactions are available even in local/dev compose.

```
Client → POST /credits → API → register_credit (Mongo)
                          → apply_async(task_id) → RabbitMQ
                          ← 202 { task_id }
Worker ← process_credit(task_id) ← RabbitMQ
       → apply_credit (Mongo transaction: claim + balance + settle)
Client → GET /credits/{task_id} → API → credit document
```

Two collections keep request history separate from aggregate user state:

- **credits** — one document per request; `_id` is the task id; status `pending | processed | failed`.
- **users** — one document per user; `balance` (present after at least one successful apply), `pending_count`, status `processing | processed`.

API and worker are separate processes. Each process creates and reuses its own sync `MongoClient` (module-level in `db.py` within that process). They are not a single shared client across process boundaries.

Key design decisions

Task payload is only `task_id`. The worker never trusts a queued amount. It reads `amount` from the credit document so the queue cannot diverge from what was registered.

Idempotency is a transactional claim. Correctness comes from the whole MongoDB transaction, not from the claim alone. Inside one `with_transaction`, the worker:

1. Claims the credit with `find_one_and_update` matching `_id` and `status: "pending"`, setting it to `processed`.
2. On match: `$inc` the user's `balance` by the credit amount and `pending_count` by `-1` (upsert).
3. Calls `settle_status` for that user.

Those steps commit together. The intermediate credit `processed` status is not externally durable before the balance update. If any step fails, the transaction aborts and a retry can attempt the entire operation again. A duplicate delivery finds no pending credit, returns false, and changes nothing—safe under at-least-once broker delivery.

`pending_count` gates `processed`. Concurrent credits for one user can finish in any order. The user stays `processing` while any credit is in flight (`pending_count > 0`). `processed` means every submitted credit for that user has reached a terminal state (`processed` or `failed`), not that at least one credit succeeded. `settle_status` therefore sets the user to `processed` when `pending_count == 0`, whether or not a `balance` field exists. Failed credits remain observable on the credits collection; an all-failed user is still settled to `processed` once nothing is pending.

Money is `Decimal` / `Decimal128`, never float. Amounts are validated to two decimal places at the API edge and stored as Decimal128 in Mongo.

Sync PyMongo, sync FastAPI endpoints, no Celery result backend. Credit status lives in Mongo (`GET /credits/{task_id}`), so a result backend would duplicate state.

Broker reliability (precise). RabbitMQ is the durable broker. Celery is configured with `task_acks_late`, `worker_prefetch_multiplier=1`, and `task_reject_on_worker_lost` so an interrupted worker can redeliver rather than silently drop work. Late acks and prefetch alone are not a complete "never lose a task" guarantee; redelivery plus the transactional claim is what makes duplicate or restarted execution safe.

Publish failure returns 503 (unknown). If `apply_async` raises `kombu.OperationalError`, the API calls `fail_credit` when possible and returns 503. The broker may have accepted the message before the error, so 503 means "unknown," not "definitely not applied."

`fail_credit` only transitions a credit from `pending` to `failed` inside a transaction (and only then decrements `pending_count` and settles). If the worker has already committed the credit as `processed`, the failure transition is a no-op. If failure wins first, a subsequently delivered task's claim is also a no-op. Only one terminal transition wins.

Without client idempotency keys, a client retry after an ambiguous 503 can register a second credit. That is an acknowledged limitation. A transactional outbox (and client keys) is the real fix and stays out of scope for this exercise.

Registration-to-publish window. The credit is inserted before publish. A crash between those steps leaves a pending credit that was never queued; a reconciler would sweep that in production. It is called out here rather than papered over with extra machinery.

Implementation approach

Layout keeps a thin surface: `config`, `logging`, `db`, `schemas`, `repository`, `api`, `celery_app`, `tasks`. Only `repository` touches collections; multi-document writes use `with_transaction`.

`POST /credits` generates a UUID `task_id`, calls `register_credit` (pending credit + user upsert with `pending_count + 1` and status `processing`), then publishes with that same id as Celery's `task_id`. `CreditTask` retries on `PyMongoError` with backoff and jitter; after retries are exhausted, `on_failure` calls `fail_credit`. Because `fail_credit` only decrements `pending_count` when it successfully moves a credit from `pending` to `failed`, repeated failure handling cannot corrupt the count.

Compose runs Mongo as `rs0` (initiated with host `mongo:27017`), RabbitMQ, one API, and one worker from a single Python 3.12 image.

Testing strategy

Tests use a real Mongo database named `credits_test` (asserted in fixtures before drops)—no Mongo mocks. Coverage is intentionally narrow and high-signal:

- **Schemas** — reject zero/negative/over-precise/non-numeric amounts and bad `user_id`s; accept values like `10.50`.
- **Repository** — create/accumulate balance; duplicate `task_id` credits once; `pending_count` keeps the user processing until the last terminal credit; all-failed user settles to `processed` when `pending_count` hits 0; `fail_credit` claim semantics (second fail is a no-op); concurrency via `ThreadPoolExecutor` proving exact sum and settled state.
- **Tasks** — apply once; second run is a no-op credit; `on_failure` marks failed only after retries are exhausted.
- **API** — publish is monkeypatched at the edge; assert 202 + pending state, 422 validation, 503 on broker `OperationalError`, GET 404/body, and `/health`.

A worker-restart smoke test is a reasonable basic check for redelivery; it does not prove every crash-recovery scenario. Killing a worker mid-transaction after a successful claim is not a reliable integration check without instrumentation, because claim, balance update, and settle share one transaction.

Out of scope

No outbox, Redis, Celery result backend, users API, or extra orchestration. Trade-offs above explain why those stay future work.